COURSE PROJECT REPORT

ON

AI-DRIVEN DUAL-MODULE SYSTEM FOR COATED-FERTILIZER NUTRIENT RELEASE PREDICTION AND CROP FERTILIZER RECOMMENDATION

Submitted in partial fulfillment of the requirements for the course

DATA SCIENCE PROGRAMMING (CSI3001)

Submitted by

Student Name	Register Number
Mohammed Afaan Sherief	24MID0178
Amit Kumar	24MID0155
Bhumi Kumari	24MID0216

Under the guidance of

Padma Priya M

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

Vellore Institute of Technology

Academic Year: 2026–2027

TABLE OF CONTENTS

Abstract

1. Introduction

2. Problem Statement

3. Objectives

4. Literature Review

5. Dataset Description

6. Methodology

7. Implementation

8. Results and Discussion

9. Conclusion and Future Scope

References

Appendix

ABSTRACT

Only 30–50% of the nitrogen applied to fields is taken up by crops. Polymer-coated (controlled-release) urea was developed to close this gap, but its release rate depends strongly on soil temperature and moisture, so a product rated for 90 days in the laboratory can be exhausted in about 40 days in a hot, wet field or last well over 120 days in a cool, dry one. Existing fertilizer recommendation systems ignore these release kinetics. This project builds an AI-driven dual-module system in Python that links the two. Module 1 predicts the field release rate and cumulative release of polymer-coated urea from soil temperature, soil moisture, rainfall, soil pH and coating thickness. Module 2 combines this prediction with crop type, growth stage, soil N-P-K and pH to recommend the fertilizer type, application amount and timing. Since no public dataset records coated-fertilizer release under field weather, 2,200 real records from the Kaggle Crop Recommendation dataset were expanded into 11,000 scenarios whose release was simulated from published kinetics. Using scikit-learn and XGBoost, Module 1 reached a test R^2 of 0.981 for release rate (Polynomial Regression) and 0.987 for cumulative release (Random Forest). Module 2 (XGBoost) selected the fertilizer type with 85.7% accuracy against an 88.4% ceiling set by label noise, and predicted amount and timing with R^2 of 0.992 and 0.976. For coated urea, the system reduced application-timing error by 82% compared with the manufacturer's static rating (22.4 to 4.0 days). The results show that feeding predicted release behaviour into the recommender produces fertilizer plans better synchronized with crop demand.

1. INTRODUCTION

A growing global population makes food security a pressing concern and calls for sustainable intensification of agriculture. Chemical fertilizers are a primary means of achieving high yields, yet they are used very inefficiently: only an estimated 30–50% of applied nutrients are taken up by the crop. The remainder is lost through leaching, runoff and volatilization, which lowers farm profitability and pollutes groundwater and surface water.

Coated fertilizers, also called controlled-release fertilizers (CRFs), were developed to release nutrients gradually so that supply follows crop uptake [2], [4]. In practice, the release pattern rarely matches the laboratory label. Release from a polymer-coated granule is driven by water diffusion through the coating, and this process is highly temperature dependent: a 10 °C rise roughly halves

the release time [15]. A coated urea rated for 90 days at 25 °C may therefore release its nitrogen in about 50 days in a hot, wet season and take 120 days or more in a cool, dry one.

Motivation. Farmers have no tool to estimate how a coated fertilizer will actually behave in their own field, and most recommendation systems do not consider release kinetics at all. As a result, fertilizer is often applied at the wrong time, and growers over-apply as insurance against losses.

Importance. Matching nutrient release to crop demand raises nutrient use efficiency, improves yield, reduces nitrogen and phosphorus losses to the environment and lowers input costs.

Scope. The project covers polymer-coated urea (three coat grades) and four conventional products (uncoated urea, DAP, MOP and NPK 19-19-19) for the 22 crops in the Kaggle Crop Recommendation dataset. It includes dataset construction, exploratory analysis, training and comparison of regression and classification models, an integrated two-stage inference pipeline, a command-line tool and a Streamlit dashboard. Field validation with measured release data is outside the scope and is listed as future work.

2. PROBLEM STATEMENT

Fertilizer recommendations today are static. A soil test returns a single application rate, and coated fertilizers are labelled with a fixed release period measured in the laboratory at 25 °C. Neither accounts for the soil temperature, moisture and rainfall of a specific field, which determine how fast a coated product actually releases its nutrients. The result is a mismatch between nutrient supply and crop demand: nutrients arrive too early or too late, efficiency drops and more fertilizer is needed for the same yield.

Problem: Given the environmental conditions of a field, the soil nutrient status and the crop's growth stage, predict how a coated fertilizer will release its nutrients and use that prediction to recommend which fertilizer to apply, how much, and when the next application is due.

Why a data-driven solution is needed. Release depends on several interacting, non-linear factors (exponential temperature effect, moisture threshold, coating thickness, elapsed time), and the best recommendation depends on these together with crop, stage and soil N-P-K. Fixed rules or a single lab rating cannot capture these interactions across thousands of field conditions, while machine learning models can learn them from data and generalize to new fields in milliseconds.

3. OBJECTIVES

1. To build a dataset that couples 2,200 real soil and climate records with literature-based release kinetics of polymer-coated urea.
2. To perform exploratory data analysis of the real and generated data to understand nutrient distributions, correlations and the spread of field release durations.
3. To develop Module 1, a regression model that predicts the release rate (% per day) and cumulative release (%) of coated urea from soil temperature, moisture, rainfall, pH, coating thickness and days since application.
4. To develop Module 2, a recommendation engine that uses Module 1's prediction together with crop, growth stage, soil N-P-K and pH to recommend fertilizer type, amount (kg/ha) and timing (days).
5. To compare Linear, Polynomial, Decision Tree, Random Forest and XGBoost models using cross-validation and appropriate metrics (R^2, RMSE, MAE, accuracy, F1-score).
6. To quantify the benefit of the dual-module design through an ablation without Module 1 and a comparison against the existing static manufacturer rating.
7. To deploy the integrated system as a command-line tool and an interactive Streamlit dashboard.

4. LITERATURE REVIEW

Recent literature falls into three related streams. The first applies machine learning to nutrient recommendation, using models such as Random Forest, Support Vector Machines and XGBoost to predict yield and compute N-P-K rates [1], [11], [12]. These systems assume that applied nutrients become available on a fixed schedule. The second stream uses IoT and sensors to capture soil moisture, temperature and electrical conductivity at high resolution [9], [11], but most studies stop at data collection and visualization rather than feeding the data into an agronomic model. The third stream models nutrient release from coated fertilizers, usually in the laboratory, with kinetic models derived from diffusion and degradation laws [2], [3], [15]. These describe release well under controlled conditions but do not account for changing field conditions, and recent reviews identify lab-to-field integration as an open question [5], [14].

Table 4.1: Summary of related work

Author and Year	Method / Technique	Key Findings	Research Gap
Brown, Luebs and Pratt (1966) [15]	Laboratory incubation of resin-coated urea at different temperatures and coating thicknesses	Release rate rises sharply with temperature and falls with coating thickness	Laboratory conditions only; no link to field weather or to recommendations
Kassem et al. (2024) [2]	Review of polymer-coated slow/controlled-release granular fertilizers	Coating material, thickness, temperature and moisture govern release; sigmoidal release curves are typical	Release models are not connected to field data or decision support
Swain et al. (2024) [3]	Diffusion-based prediction model of release from single coated granules	Coating properties can be used to predict diffusion release of a granule	Single-granule, controlled conditions; environmental variability not modelled
Tanaka et al. (2024) [1]	Machine learning models for site-specific fertilizer application recommendations	Accurate yield prediction does not automatically give accurate fertilizer-rate recommendations	Treats fertilizer as immediately available; release kinetics ignored
Wu et al. (2024) [12]	ML forecasting of fertilizer components from physical soil parameters	Soil physical parameters improve prediction of fertilizer requirements	No modelling of coated products or application timing
Padhiary (2024) [11]	Review of ML and AI in precision farming operations	ML and IoT enable data-driven, site-specific nutrient management	Most work stops at sensing and prediction; integration with release behaviour is missing

Identified gap. No integrated system uses environmental data to predict how a coated fertilizer will release its nutrients in a specific field and then feeds that prediction into a recommendation engine [6], [10]. This project addresses that gap with a dual-module architecture.

5. DATASET DESCRIPTION

The project combines real field data with a physics-based simulation, because no public dataset records coated-fertilizer release under field weather. Table 5.1 lists the four datasets used.

Table 5.1: Datasets used in the project

Dataset	Source	Size	Content
Crop Recommendation	Real (Kaggle) [16]	2,200 × 8	Soil N, P, K, pH, air temperature, humidity, rainfall; 22 crops, 100 records each
Crop N-P-K requirements	Real (Harvestify) [17]	22 × 6	Target N, P, K per crop

Dataset	Source	Size	Content
Nutrient release dataset	Generated (Module 1)	11,000 × 11	Field conditions, coating thickness, days → release rate, cumulative release
Fertilizer recommendation dataset	Generated (Module 2)	11,000 × 14	Crop, stage, N-P-K, pH, release rate → fertilizer type, amount, timing

5.1 Attributes

Table 5.2: Main attributes

Attribute	Unit	Description
N, P, K	kg/ha	Initial soil nitrogen, phosphorus and potassium (real)
soil_pH	–	Soil pH (real, with ±0.1 noise)
soil_temperature	°C	Air temperature + 1–6 °C, clipped to 8–48 °C
soil_moisture	% vol.	Derived from humidity and rainfall, clipped to 5–50%
rainfall	mm	Seasonal rainfall (real, with log-normal variation)
coating_thickness	µm	Polymer coat thickness, 20–100 µm
days_since_application	days	0–150 days
crop_type, growth_stage	category	22 crops; sowing, vegetative, flowering, maturity
release_rate (target, M1)	% per day	Mean release rate over the main release phase
cumulative_release (target, M1)	%	Share of nutrient released after the given days
fertilizer_type (target, M2)	7 classes	Urea, PCU-60, PCU-90, PCU-120, DAP, MOP, NPK 19-19-19
fertilizer_amount (target, M2)	kg/ha	Product mass to apply
application_timing (target, M2)	days	Days until the next application

5.2 Data quality and preprocessing

- **Missing values and duplicates:** none were found in the real data or in either generated dataset.
- **Outliers:** the IQR rule flags high P, K and rainfall values (for example apple and grape soils with K ≈ 200). These are genuine crop-specific values and were kept; derived soil values were clipped to physical ranges.
- **Encoding and scaling:** crop type (22), growth stage (4) and fertilizer type (7) were one-hot encoded; fertilizer classes were label-encoded for XGBoost. Features were standardized for linear models; tree models used raw values.

- **Realism:** 6% granule-batch variability, 3% measurement noise on release rate, ±1.5% noise on cumulative release and 3% random label changes (agronomist disagreement) were added so the learning problem is not trivial.

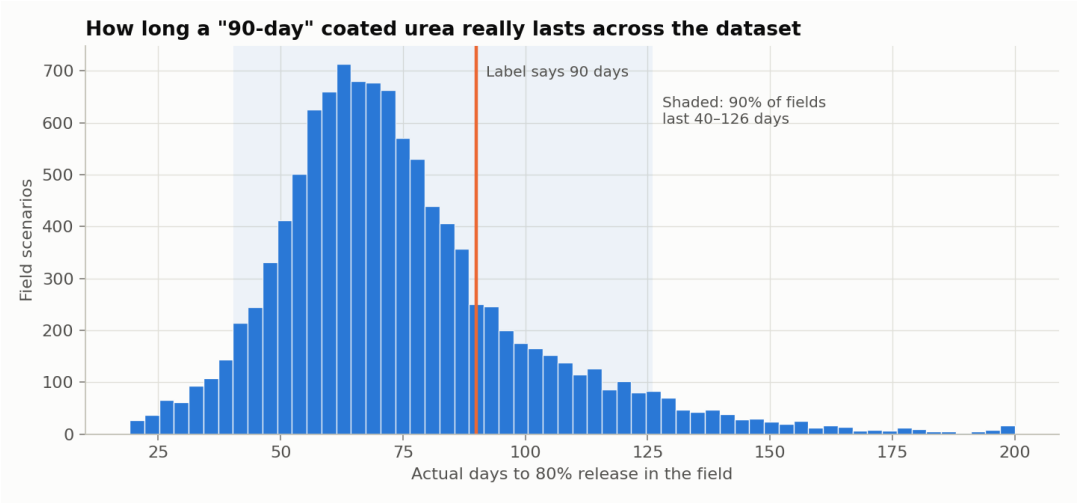


Figure 5.1: Simulated field release duration of the "90-day" coated urea across 11,000 scenarios

Figure 5.1 shows that the same "90-day" product actually lasts 40–126 days across the dataset (5th–95th percentile, median 70 days), reproducing the 50–120-day field swing reported in the literature and motivating the project.

6. METHODOLOGY

The workflow (Figure 6.1) has five stages: data collection, data generation and preprocessing, Module 1 training, Module 2 training with Module 1's predictions as input, and evaluation against a static baseline.

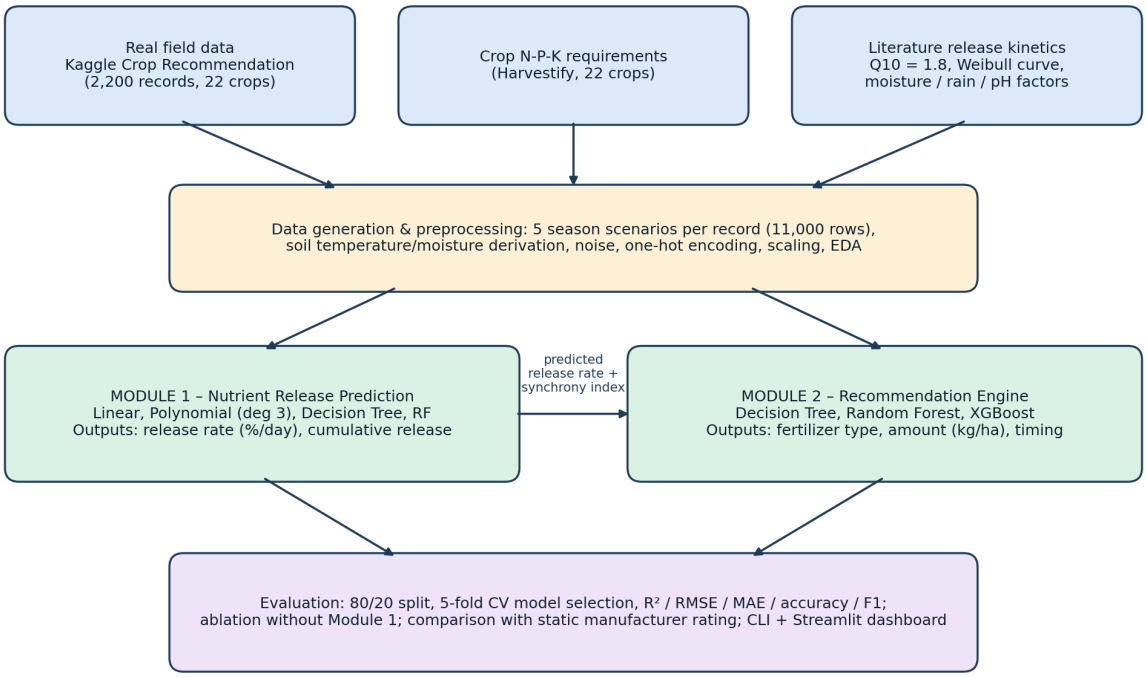


Figure 6.1: Project workflow of the dual-module system

6.1 Release simulation

Each real record was expanded into five season scenarios. The manufacturer-rated duration (days to 80% release at 25 °C) grows linearly with coating thickness L, so a 65 µm coat gives the standard 90-day product:

$$D_{25} = 1.3\,L + 5.5 \qquad (1)$$

Field conditions scale the release rate by a factor k, where Q10 = 1.8 reflects the finding that a 10 °C rise roughly halves release time [15], f_M lowers release in dry soil, f_R raises it slightly under heavy rainfall, and f_pH = 1 + 0.04|pH − 6.5| adds a weak pH effect:

$$k = Q10^{((T-25)/10)} \cdot f_M(\theta) \cdot f_R(R) \cdot f_{pH} \qquad (2)$$

The field duration, mean release rate and Weibull-shaped cumulative release (shape β = 1.8, giving the lag, release and plateau phases typical of polymer-coated urea) follow as:

$$D = D_{25} / k, \quad r = 80 / D \qquad (3)$$

$$F(t) = 100\,[1 - \exp(-(t/\tau)^{\beta})], \quad \tau = D / (\ln 5)^{(1/\beta)} \qquad (4)$$

6.2 Recommendation labels

Labels for Module 2 were produced by agronomic rules. The most limiting nutrient (largest relative deficit against the crop target) decides the product: DAP for phosphorus, MOP for potassium and NPK 19-19-19 when nutrients are co-limiting or no deficit is clear. For nitrogen, uncoated urea is chosen late in the season; otherwise the coated urea grade (PCU-60, PCU-90 or PCU-120) whose field duration best matches the crop's remaining demand window is chosen. The amount is the nutrient need divided by the product's nutrient content and a use efficiency that falls when release and demand are out of step; timing is the number of days until the next application.

6.3 Models and justification

- **Linear and Polynomial Regression (Module 1):** interpretable baselines for a physical relationship; degree-3 polynomial terms let the linear model follow the exponential temperature effect.
- **Decision Tree (Module 1, Module 2 baseline):** captures non-linear interactions such as "high temperature and moist soil imply fast release" and gives feature importance. Module 1 tree depth and leaf size were tuned by grid search.
- **Random Forest:** averages many trees to reduce variance and overfitting; used as a benchmark in Module 1 and a candidate in Module 2.
- **XGBoost (Module 2):** gradient-boosted trees that handle mixed categorical and numerical inputs with complex interactions, which suits the recommendation task [19].
- **Synchrony index:** an engineered feature equal to the predicted release duration divided by the days of crop demand remaining, passed from Module 1 to Module 2.

6.4 Evaluation protocol

All models used an 80/20 train–test split (stratified by fertilizer type for Module 2). The best model for each target was selected by mean 5-fold cross-validation score on the training set, and the held-out test set was used once for final evaluation. Regression was evaluated with MSE, RMSE, MAE and R^2; classification with accuracy, macro-F1 and the confusion matrix. Two further comparisons test the central claim: an ablation of Module 2 without Module 1's features, and the existing static system that uses the manufacturer's 25 °C rating ($k = 1$).

7. IMPLEMENTATION

Programming environment and tools:

Tool	Use in the project
Python 3.9	Programming language for data processing and modelling
pandas 2.3, NumPy 2.0	Data manipulation and generation of the simulated release data
scikit-learn 1.6	Linear/Polynomial Regression, Decision Tree, Random Forest, pipelines, cross-validation, metrics [18]
XGBoost 2.1	Gradient-boosted classifier and regressors for Module 2 [19]
Matplotlib 3.9	Exploratory and result visualizations
joblib	Saving and loading trained models
Streamlit 1.50, Altair 5.5	Interactive web dashboard
pytest 8.4	13 automated tests of the physics, datasets and integrated system
Jupyter Notebook, VS Code	Development and executed walkthrough notebook

The code is organized into modules under src/: release_physics.py (release kinetics), agronomy.py (recommendation rules), generate_dataset.py, eda.py, train_module1.py, train_module2.py and pipeline.py (integrated inference). main.py runs the whole pipeline end to end. The key parts are shown below.

7.1 Release kinetics (src/release_physics.py)

This function implements Equation (2). Each factor multiplies the laboratory release rate; a value of 1.0 corresponds to lab conditions at 25 °C and 30% moisture.

```
def rate_factor(soil_temperature, soil_moisture, rainfall, soil_pH):
    """Multiplier on the lab release rate under field conditions (1.0 = lab)."""
    f_temp = Q10 ** ((np.asarray(soil_temperature) - T_REF) / 10.0)
    f_moist = 0.5 + 0.5 * (1.0 - np.exp(-np.asarray(soil_moisture) / 12.0))
    f_moist = f_moist / (0.5 + 0.5 * (1.0 - np.exp(-30.0 / 12.0)))
    f_rain = 1.0 + 0.12 * np.tanh((np.asarray(rainfall) - 100.0) / 80.0)
    f_ph = 1.0 + 0.04 * np.abs(np.asarray(soil_pH) - 6.5)
    return f_temp * f_moist * f_rain * f_ph

def cumulative_release(days, duration_days):
    tau = np.asarray(duration_days) / np.log(5.0) ** (1.0 / WEIBULL_SHAPE)
    return 100.0 * (1.0 - np.exp(-(np.asarray(days) / tau) ** WEIBULL_SHAPE))
```

7.2 Dataset generation (src/generate_dataset.py)

Each real field record is repeated for five season scenarios. Soil temperature and moisture are derived from the real air temperature, humidity and rainfall, and noise is added so the data resembles field measurements.

```
df = fields.loc[fields.index.repeat(SCENARIOS_PER_FIELD)].reset_index(drop=True)
df["soil_temperature"] = np.clip(df["temperature"] + rng.uniform(1, 6, n)
                                 + rng.normal(0, 1.5, n), 8, 48)
df["soil_moisture"] = np.clip(6 + 0.22 * df["humidity"] + 0.06 * df["rainfall"]
                              + rng.normal(0, 3, n), 5, 50)
batch = rng.lognormal(0, 0.06, n)   # granule-to-granule coating variability
duration = phys.effective_duration(df["coating_thickness"], df["soil_temperature"],
            df["soil_moisture"], df["rainfall"], df["soil_pH"]) * batch
```

7.3 Module 1 training (src/train_module1.py)

Four candidate regressors are trained for each target. The Decision Tree is tuned with GridSearchCV, and the final model is selected on 5-fold cross-validated R^2, not on the test set, to avoid optimistic bias.

```
def candidate_models():
    return {
        "Linear Regression": make_pipeline(StandardScaler(), LinearRegression()),
        "Polynomial LR (deg 3)": make_pipeline(StandardScaler(), PolynomialFeatures(3),
                                               LinearRegression()),
        "Decision Tree": GridSearchCV(DecisionTreeRegressor(random_state=SEED),
            {"max_depth": [6, 10, 14, None], "min_samples_leaf": [1, 5, 10]},
            cv=5, scoring="neg_mean_squared_error"),
        "Random Forest (benchmark)": RandomForestRegressor(n_estimators=300, ...),
    }
...
best = max(results, key=lambda n: results[n]["CV_R2_mean"])  # select on CV, not test
```

7.4 Module 2 pipeline and integration (src/train_module2.py)

Categorical features are one-hot encoded inside a scikit-learn Pipeline so the same preprocessing is applied at training and inference. The release rate predicted by Module 1 for the reference 90-day coat is added to every row before Module 2 is trained, which is the link between the two modules.

```
def make_pipeline(model, numeric, categorical=M2_CATEGORICAL):
    pre = ColumnTransformer([
        ("cat", OneHotEncoder(handle_unknown="ignore", sparse_output=False), categorical),
        ("num", "passthrough", numeric)])
    return Pipeline([("prep", pre), ("model", model)])
```

```
def add_module1_predictions(df):
    m1 = joblib.load(MODEL_DIR / "module1_release_rate.joblib")
    rate = m1["model"].predict(df.assign(coating_thickness=REFERENCE_THICKNESS)[m1["features"]])
    return df.assign(**agronomy.module1_features(rate, df["crop_type"], df["growth_stage"]))
```

7.5 Integrated inference (src/pipeline.py)

At inference, SmartFertilizerSystem.recommend() checks inputs against the training range, runs Module 1, builds the Module 2 feature row, predicts the product with its confidence, and then predicts the amount and timing for that product.

```
release = self.predict_release(field, days=days_since_application)        # Module 1
m1 = agronomy.module1_features([release["release_rate"]], [crop_type], [growth_stage])
X = pd.DataFrame([{"crop_type": crop_type, "growth_stage": growth_stage,
                   "N": N, "P": P, "K": K, "soil_pH": field["soil_pH"], **m1}])
proba = type_pipe.predict_proba(X[rec["categorical"] + rec["numeric"]])[0]  # Module 2
product = ranked[0][0]
X = X.assign(fertilizer_type=product)
amount = rec["pipelines"]["fertilizer_amount"].predict(X)[0]
timing = rec["pipelines"]["application_timing"].predict(X)[0]
```

The system can be run with a single command (python3 run_project.py), which sets up the environment, trains the models on first run and opens the Streamlit dashboard. predict.py offers the same recommendation from the command line.

8. RESULTS AND DISCUSSION

8.1 Exploratory data analysis

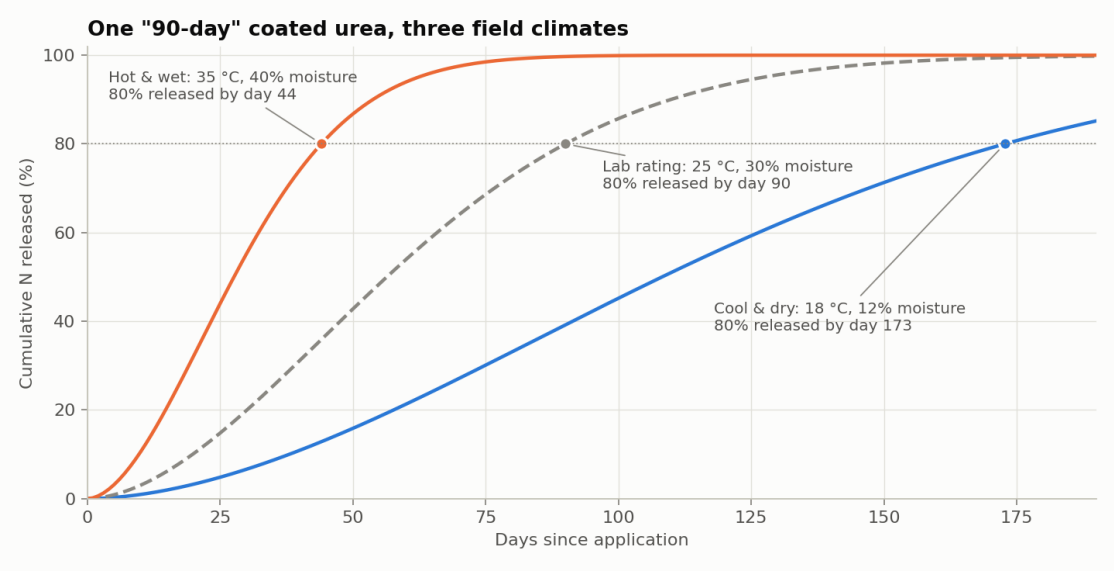


Figure 8.1: Cumulative release of the same "90-day" coated urea under three field climates

Figure 8.1 shows the S-shaped release curves of a single product under different climates. In a hot, wet field 80% of the nitrogen is released far earlier than the label suggests, while in a cool, dry field release is much slower. This confirms that the label alone cannot be used to time fertilizer applications.

8.2 Module 1: nutrient release prediction

Table 8.1: Module 1 test-set performance (2,200 test rows)

Model	Rate R^2	Rate RMSE (%/day)	Cumulative R^2	Cumulative RMSE (%)
Linear Regression	0.811	0.396	0.809	15.04
Polynomial LR (deg 3)	**0.981**	**0.124**	0.960	6.88
Decision Tree (grid-searched)	0.960	0.182	0.966	6.38
Random Forest (benchmark)	0.977	0.138	**0.987**	**3.96**

Plain Linear Regression underfits (R^2 0.811) because temperature affects release exponentially. Adding degree-3 polynomial terms raises R^2 to 0.981, and this model was selected for release rate (CV R^2 0.979). For cumulative release, which also depends on elapsed time, the Random Forest

was selected with test R^2 0.987 and RMSE 3.96 percentage points. MAE was 0.087 %/day for release rate and 2.66% for cumulative release.

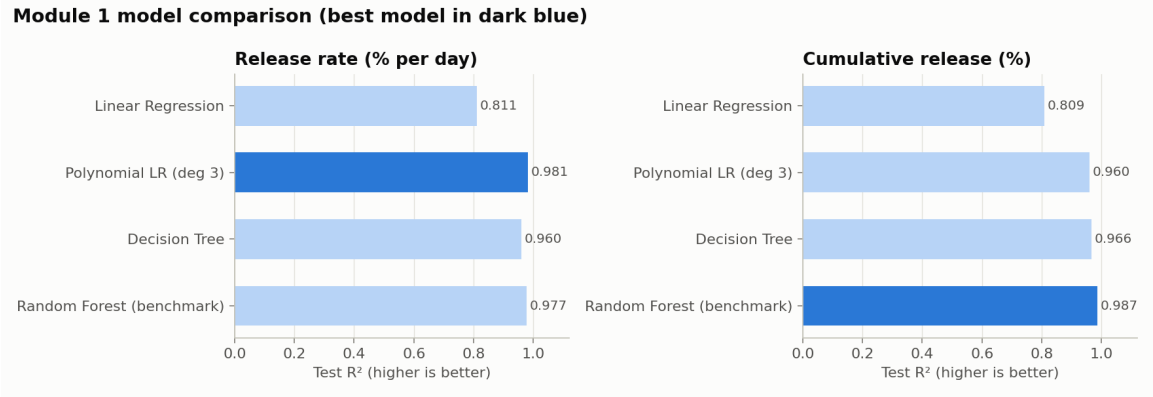


Figure 8.2: Module 1 test R^2 by model

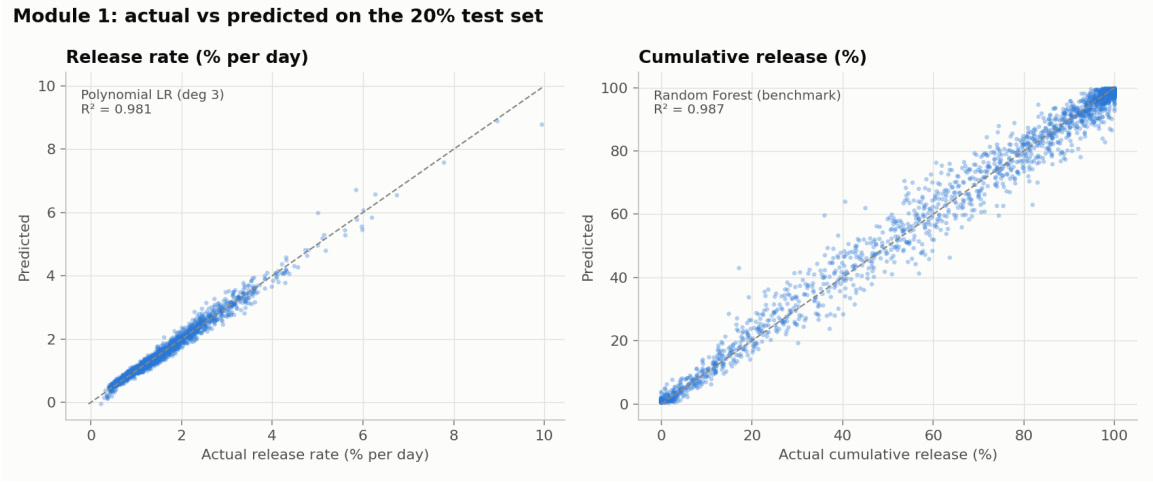


Figure 8.3: Actual vs predicted values of the selected Module 1 models

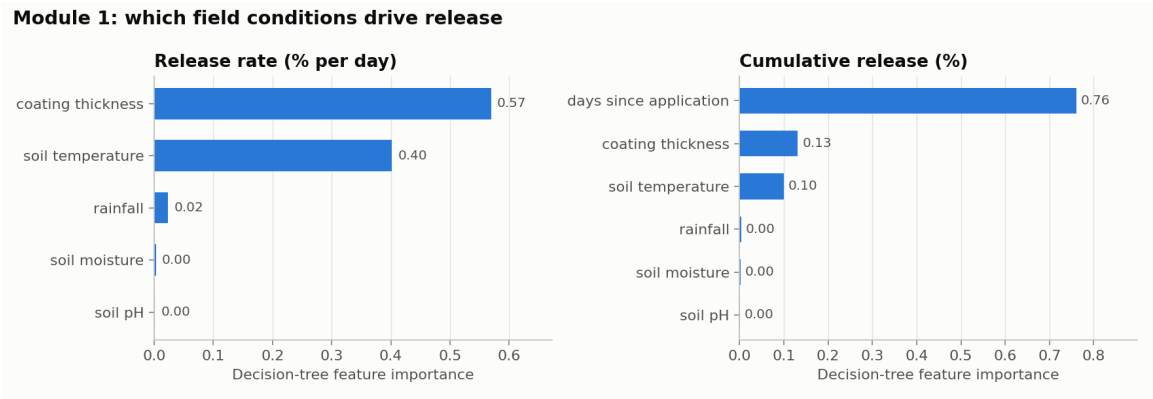


Figure 8.4: Decision-tree feature importance for Module 1

Points in Figure 8.3 lie close to the diagonal, showing good agreement across the full range. Feature importance (Figure 8.4) matches the underlying physics: coating thickness (57%) and soil

temperature (40%) explain almost all of the release-rate variation, while days since application dominates cumulative release (76%).

8.3 Module 2: fertilizer recommendation

Table 8.2: Module 2 test-set performance

Model	Type accuracy	Macro-F1	Coated-urea accuracy	Amount R^2	Timing R^2
Decision Tree	80.4%	0.786	75.3%	0.987	0.961
Random Forest	83.0%	0.811	78.5%	0.992	0.973
XGBoost (selected)	**85.7%**	**0.839**	**80.4%**	**0.992**	**0.976**
XGBoost without Module 1	83.6%	0.789	71.0%	0.991	0.904

XGBoost achieved the highest 5-fold CV score for all three targets (type CV accuracy 85.5%) and was selected. Its test accuracy of 85.7% should be read against the 88.4% achieved by the exact labelling rule itself, which is the ceiling given the injected label noise; the model is therefore close to the best achievable. Given the recommended product, amount and timing reach R^2 of 0.992 (MAE 8.4 kg/ha) and 0.976 (MAE 2.4 days). End to end, when the product itself is predicted, R^2 is 0.76 for amount and 0.83 for timing, because a wrong product implies a different dose.

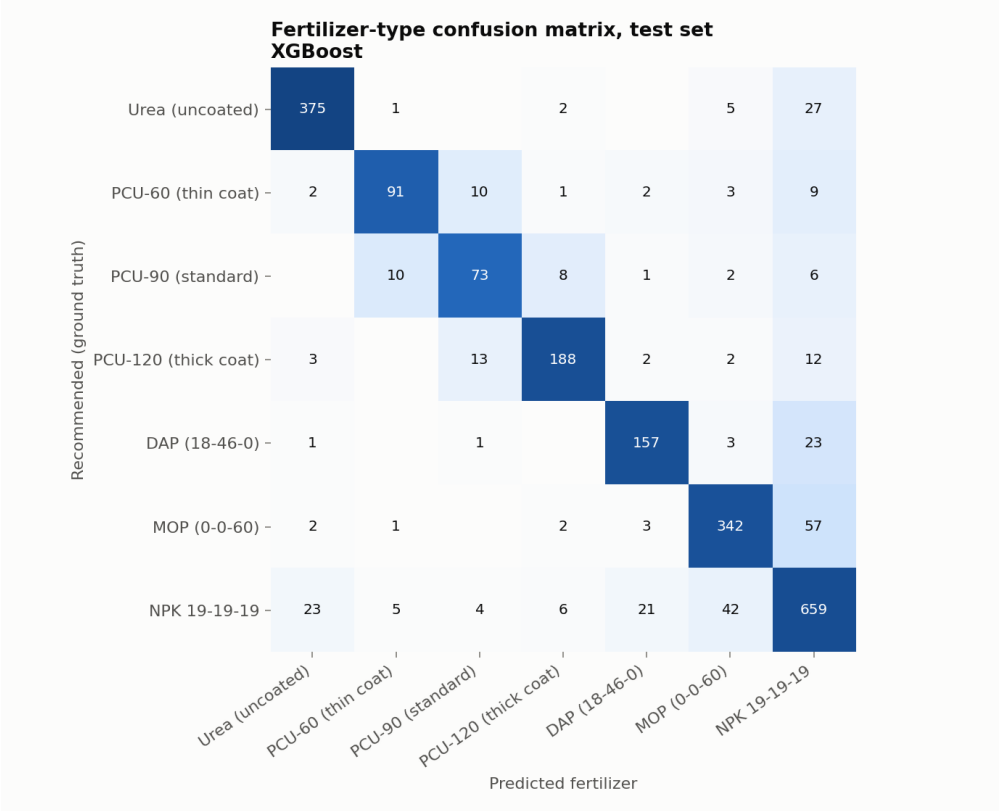


Figure 8.5: Confusion matrix of the XGBoost fertilizer-type classifier

The confusion matrix shows that most errors fall between neighbouring coat grades (for example PCU-90 vs PCU-120) or involve the general NPK product, which are also the cases where agronomists would most often disagree.

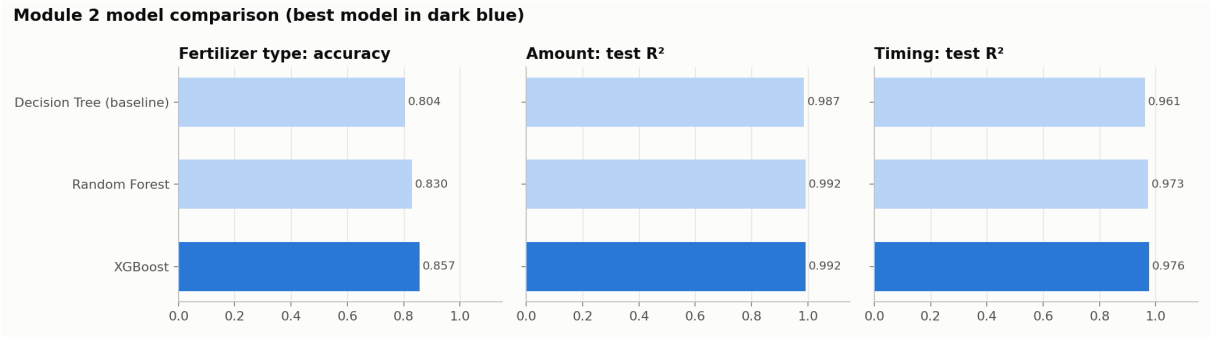


Figure 8.6: Module 2 model comparison

8.4 Comparison with the existing static system

Table 8.3: Mean absolute error of the three systems on the test set

System	Timing MAE, all (days)	Timing MAE, coated urea (days)	Amount MAE (kg/ha)
Existing static system (25 °C lab rating)	9.2	22.4	12.6
ML without Module 1 (soil test only)	4.4	8.5	9.0
Proposed dual-module system	**2.4**	**4.0**	**8.4**

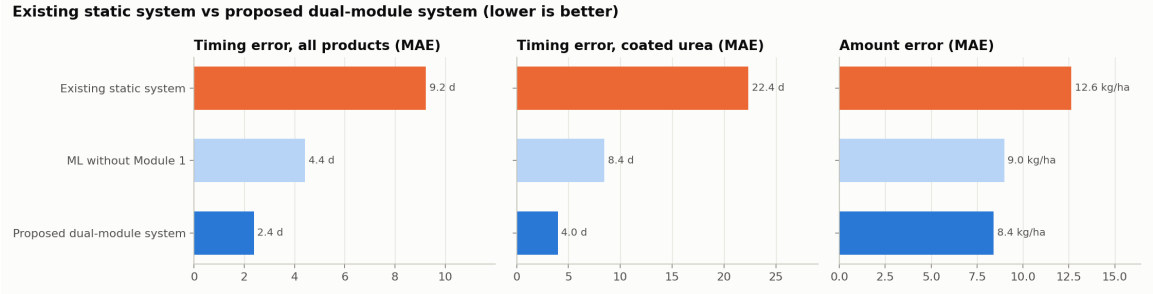


Figure 8.7: Application-timing error of the three systems

The static approach mistimes the next coated-urea application by 22.4 days on average; the proposed system reduces this to 4.0 days, an 82% reduction. The ablation confirms that the gain comes from Module 1: without the predicted release rate, coated-grade accuracy falls from 80.4% to 71.0% and timing R^2 from 0.976 to 0.904.

8.5 Integrated pipeline on user inputs

Table 8.4: Sample system inputs and outputs

	Hot, wet rice field	Cool, dry maize field
Soil temperature / moisture	36 °C / 42%	18 °C / 14%
Rainfall / soil pH	240 mm / 6.2	45 mm / 6.8
Growth stage; soil N-P-K (kg/ha)	Sowing; 62-45-38	Sowing; 62-48-22
Module 1: release rate of 90-day PCU	1.93% per day	0.56% per day
Module 1: actual release duration	≈ 41 days	≈ 144 days
Module 2: recommended type	PCU-120, thick coat (71%)	PCU-60, thin coat (94%)
Module 2: amount	246 kg/ha	223 kg/ha
Module 2: next application in	46 days	92 days

In the hot, wet rice field Module 1 predicts that the standard 90-day product would be exhausted in about 41 days, so Module 2 recommends the thickest coat. In the cool, dry maize field the same product would last about 144 days, so the thinnest coat is recommended. The two-stage process runs correctly from raw inputs to an actionable fertilizer plan, both from the command line and in the Streamlit dashboard.

9. CONCLUSION AND FUTURE SCOPE

This project built a dual-module AI system that reconnects static laboratory ratings of coated fertilizers with the dynamic conditions of the field. All objectives were met: a coupled real-plus-simulated dataset was built, Module 1 predicted field release with R^2 of 0.98–0.99, and Module 2 (XGBoost) recommended the product with 85.7% accuracy (close to the 88.4% ceiling) and sized the dose and timing with $R^2 \geq 0.97$. Compared with the manufacturer's static rating, the system reduced coated-urea timing error by 82%, and the ablation showed that this improvement depends on Module 1.

Practical applications. The system can serve as a decision-support tool for farmers, agronomists and extension services to choose the right coated-fertilizer grade for a season, schedule top-dressing, and reduce over-application. Fertilizer manufacturers could use Module 1 to publish climate-specific release guidance instead of a single label.

Limitations.

- Release behaviour and recommendation labels are simulated from published relationships, so the models learn those relationships rather than measured field outcomes.
- Soil temperature and moisture are derived from air temperature, humidity and rainfall rather than measured with soil sensors.
- Recommendations for N-P-K values outside a crop's range in the real data are extrapolations (the tool flags these).

Future scope.

- Calibrate Module 1 with laboratory incubation or lysimeter release measurements for several fertilizers.

- Collect historical soil, weather, application and yield data from partner farms to train on real outcomes.
- Connect live weather forecasts and IoT soil sensors for in-season updates.
- Extend the system to sulphur-coated urea and coated phosphorus and potassium products.

REFERENCES

[1] T. S. Tanaka, G. B. M. Heuvelink, T. Mieno, and D. S. Bullock, "Can machine learning models provide accurate fertilizer application recommendations?," Precision Agriculture, vol. 25, no. 4, pp. 1839–1856, 2024.

[2] I. Kassem, E. H. Ablouh, F. Z. El Bouchtaoui, M. Jaouahar, and M. El Achaby, "Polymer-coated slow/controlled-release granular fertilizers: Fundamentals and research trends," Progress in Materials Science, vol. 144, art. no. 101269, 2024.

[3] S. Swain et al., "Proportional impact prediction model of coating material on polymer-coated fertilizers: Diffusion release from single granules," Scientific Reports, vol. 14, art. no. 53410, 2024.

[4] A. P. Dovzhenko et al., "Recent progress in the development of encapsulated controlled-release fertilizers," Heliyon, vol. 10, art. no. e12667, 2024.

[5] S. Goyil, "Controlled-release fertiliser: Recent developments and future prospects," Journal of Controlled Release, vol. 366, pp. 233–249, 2024.

[6] X. Li, "Global trends and current advances in slow/controlled-release fertilizers: A meta-analysis and synthesis," Computers and Electronics in Agriculture, vol. 220, art. no. 108456, 2024.

[7] R. I. Rubel, M. S. Alam, and T. Khan, "Economic assessment of biochar-based controlled-release fertilizers," Environmental Science and Pollution Research, vol. 32, pp. 20401–20417, 2025.

[8] N. Singh et al., "Encapsulated slow-release fertilizers: A review," Environmental Nanotechnology, Monitoring & Management, vol. 25, art. no. 100876, 2024.

[9] J. Yin, "Agroecosystem modeling and precision agriculture for fertilizer optimization," Agricultural Systems, vol. 227, art. no. 103501, 2025.

[10] T. M. C. Maaz, "A review of research and innovation on novel fertilizers for efficient nutrient management in agriculture," Nature Food, vol. 6, no. 3, pp. 155–168, 2025.

[11] M. Padhiary, "Enhancing precision agriculture: A comprehensive review of machine learning and artificial intelligence in farming operations," Computers and Electronics in Agriculture, vol. 234, art. no. 106283, 2024.

[12] M. Wu et al., "Precision forecasting of fertilizer components using machine learning and physical soil parameters," Soil & Tillage Research, vol. 214, art. no. 105469, 2024.

[13] H. S. Hameed, "A review of nano-enabled controlled-release fertilizers," Journal of Agricultural Engineering, vol. 12, pp. 45–63, 2024.

[14] A. Ravindran, “Advancements and challenges in controlled-release fertiliser: Recent developments and future prospects,” Journal of Controlled Release, vol. 380, art. no. 101345, 2025.

[15] M. J. Brown, R. E. Luebs, and P. F. Pratt, “Effect of temperature and coating thickness on the release of urea from resin-coated granules,” Agronomy Journal, vol. 58, pp. 175–178, 1966.

[16] A. Ingle, “Crop Recommendation Dataset,” Kaggle. [Online]. Available: https://www.kaggle.com/datasets/atharvaingle/crop-recommendation-dataset

[17] Harvestify, “Crop-wise fertilizer requirement data,” GitHub repository. [Online]. Available: https://github.com/Gladiator07/Harvestify

[18] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[19] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2016, pp. 785–794.

APPENDIX

A. Source code

GitHub repository: https://github.com/Amit-kr04/Smart-fertilizer-project

Project structure:

```
main.py                  run everything end to end
run_project.py           one-command setup, training and dashboard launch
predict.py               CLI recommendation for user inputs
dashboard.py             Streamlit dashboard
Smart_Fertilizer_Project.ipynb   executed walkthrough notebook
src/config.py            paths, constants, crop cycles, fertilizer products
src/release_physics.py   coated-urea release kinetics
src/agronomy.py          recommendation rules, nutrient needs, synchrony index
src/generate_dataset.py  builds both datasets from the real field data
src/eda.py               exploratory figures
src/train_module1.py     Module 1 training and evaluation
src/train_module2.py     Module 2 training, ablation, static baseline
src/pipeline.py          SmartFertilizerSystem: Module 1 -> Module 2
tests/test_project.py    13 pytest tests
models/                  trained models (.joblib)
outputs/                 figures and metrics (JSON)
```

Running the project:

```
python3 run_project.py                     # setup + train + open dashboard
.venv/bin/python main.py                   # full pipeline (~2 min)
.venv/bin/python predict.py --crop rice --stage sowing --N 62 --P 45 --K 38 \
    --temp 36 --moisture 42 --rainfall 240 --ph 6.2 --days 20
.venv/bin/python -m pytest -q              # 13 tests
```

B. Sample dataset records

Table B.1: Real data – Kaggle Crop Recommendation (first rows)

N	P	K	Temp (°C)	Humidity (%)	pH	Rainfall (mm)	Crop
90	42	43	20.88	82.00	6.50	202.94	rice
85	58	41	21.77	80.32	7.04	226.66	rice
60	55	44	23.00	82.32	7.84	263.96	rice

Table B.2: Generated nutrient release dataset (Module 1)

Crop	Soil temp	Moisture	Rainfall	Coating (µm)	pH	Days	Rated dur.	Rate (%/d)	Cum. (%)
rice	26.74	36.09	199.26	95.14	6.27	41	129.19	0.737	25.31
rice	23.57	36.51	194.52	47.89	6.57	37	67.76	1.219	42.29

Table B.3: Generated fertilizer recommendation dataset (Module 2)

Crop	Stage	N	P	K	pH	Ref. rate (%/d)	Type	Amount (kg/ha)	Timing (d)
rice	sowing	90	42	43	6.48	0.893	NPK 19-19-19	524.8	31
rice	vegetative	90	42	43	6.45	1.029	NPK 19-19-19	322.9	25

C. Additional graphs

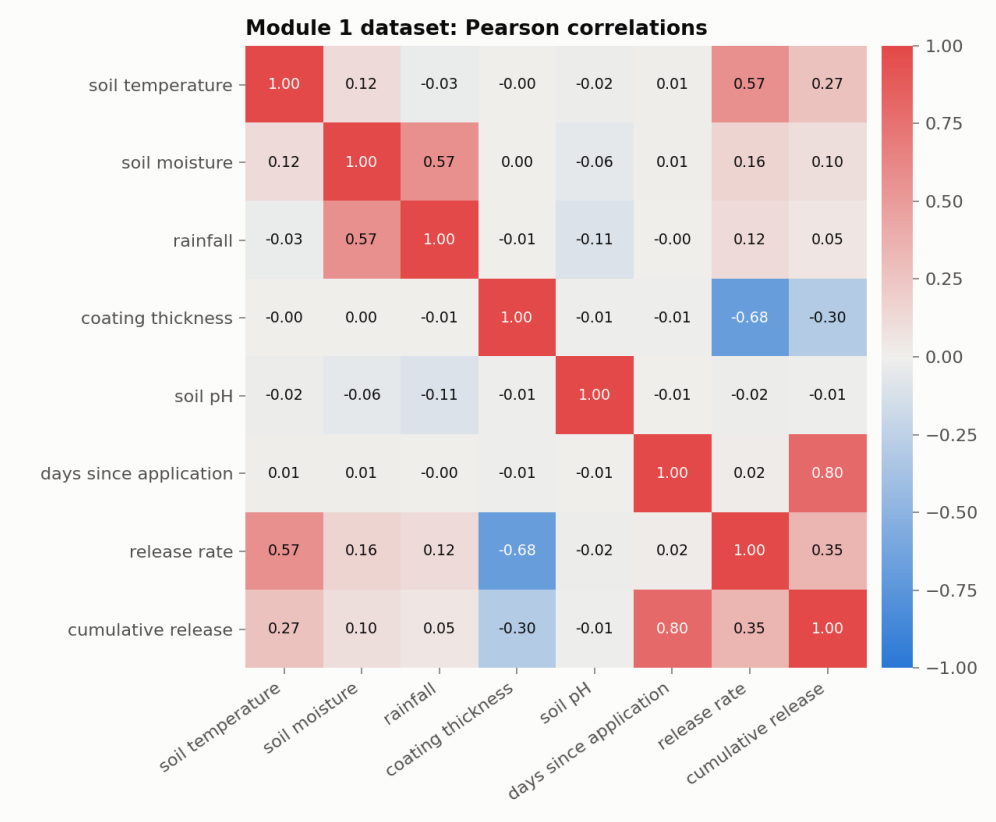


Figure C.1: Correlation heatmap of the nutrient release dataset

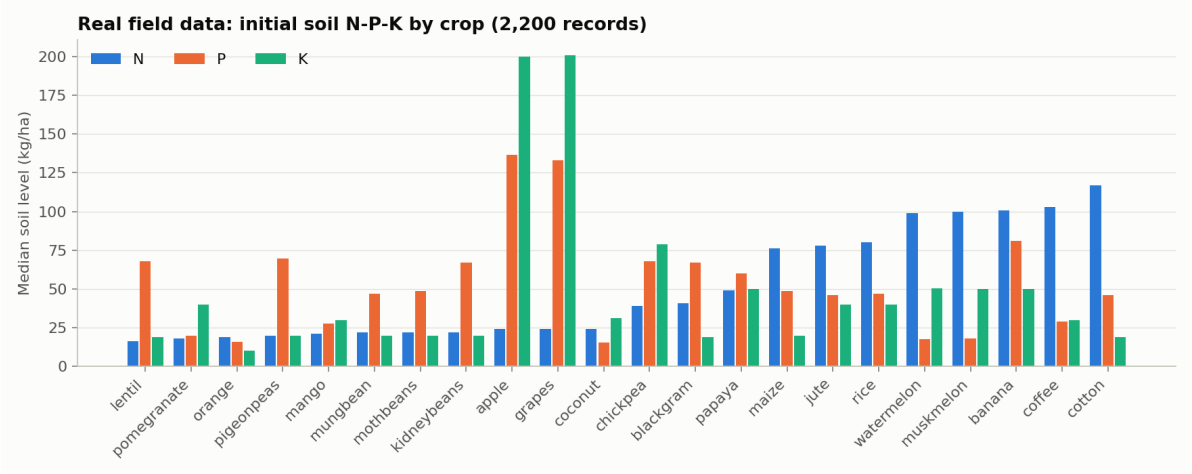


Figure C.2: Soil N-P-K by crop in the real dataset

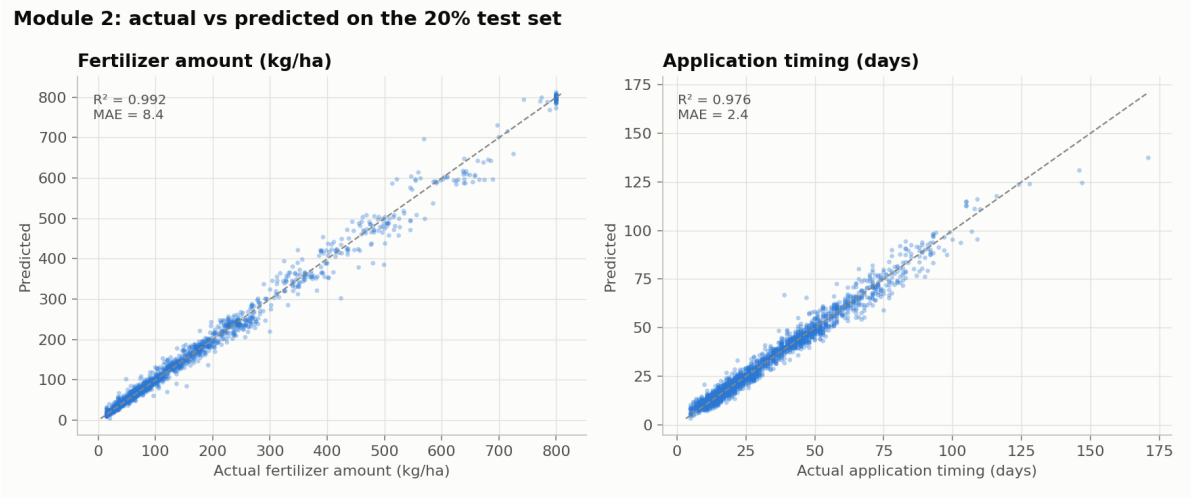


Figure C.3: Module 2 actual vs predicted amount and timing

D. Supplementary outputs

Full metrics for every model, including MSE, MAE, cross-validation means and standard deviations, end-to-end scores and the rule-ceiling accuracy, are saved in outputs/module1_metrics.json, outputs/module2_metrics.json and outputs/results_summary.json in the repository.